

Python Programming Exercises (Punch & Enbody Chap 9)

1. We consider an Iris database (<http://archive.ics.uci.edu/ml>).

Irises come in three different species: *setosa*, *virginica*, and *versicolor*. Each entry in the data set has five values: sepal length, sepal width, petal length, petal width, and the species name (*setosa*, *virginica*, and *versicolor*). Read in the data and create a dictionary with the key as the species name and the values as the averages of each of the four characteristic values: sepal length, sepal width, petal length, petal width. That is, the size of the dictionary will be three. Write functions to create the dictionary and display the contents of the dictionary.

2. Programmers are often required to take a file of text and analyze it in various ways. With dictionaries, we can now do more analysis. In particular, we can determine the word frequency of a file with the aid of a dictionary. As we did in the task during the lecture, we use each word as a key and use as a value the number of occurrences of that word. For every word in the file, we check to see whether that word is already in the dictionary. If the word is in fact already there, we increase its associated count (value) by 1. If the word is not yet in the dictionary, we insert it with a count (value) of 1.

You can achieve the overall aim by defining the following functions:

add_word: Add each word to the dictionary. Parameters are the word and a dictionary. No return value.

process_line: There is some work to be done to process the line: strip off various characters (e.g. commas, periods), split out the words, and so on (you will need some string functions, e.g. `split()`, `strip()`, etc). Parameters are a line and the dictionary. It calls `add_word` with each processed word. No return value.

main: We will use a main function below as the main program. It will open the file and call `process_line` on each line. When finished, it will call `print` to print the dictionary.

```
def main():
    word_count_dict={}
    gba_file = open('test.txt','r')
    for line in gba_file:
        process_line(line, word_count_dict)
    print('Length of the dictionary:',len(word_count_dict))
    print(word_count_dict)
```

3. Building on the work in question 1, you can use sets to discover properties such as the common words, the number of unique words used by two documents, and the number of unique words used in each document. Here is a list of functions we will reuse:

add_word: Add each word to the set. Parameters are the word and a set. No return value.

process_line: this function can be used as is. Though the parameter names are inappropriate for this context

main: The main function must now open two files to do the comparison, and for each we must create a set for processing and then process all the lines of the file using `process_line`. The main function will print the lengths of the two sets, the intersection of the two sets.